WIRELESS PROJECT REPORT

PATH LOSS MODELLING

Prateek Maheshwari (cs25m525)

April 2026

1. Problem Statement

In wireless communication systems, accurate estimation of signal attenuation between a Transmitter (Tx) and a Receiver (Rx) is critical for efficient network design, optimization, and performance evaluation. Signal strength typically decreases with distance due to factors such as path loss, shadowing, and environmental conditions, making reliable prediction challenging.

The problem addressed in this project is to develop robust and accurate path loss models that can predict the reduction in signal strength over varying distances and environments. Traditional empirical models often fail to generalize well across different scenarios, necessitating data-driven approaches.

This project aims to utilize both supervised and unsupervised machine learning techniques to analyze real-world signal data and model the relationship between distance and signal attenuation. The goal is to improve prediction accuracy, identify underlying patterns in the data, and evaluate whether a single model can generalize across environments or if environment-specific models are required.

Ultimately, the project seeks to enhance the reliability of signal strength prediction, thereby contributing to improved planning and optimization of wireless communication networks.

1.1 Objectives

- Preprocess raw measurement data and extract physically meaningful features.
- Compute empirical path loss using the log-distance model and fit the path loss exponent n.
- Train and evaluate Random Forest and Gradient Boosting regressors for path loss prediction.
- Implement hybrid models combining log-distance residuals with ML for improved accuracy.
- Compare all models on both zones using RMSE and R^2 metrics.

2. Dataset Description

The raw dataset was collected via a mobile phone app logging signalling parameters (RSRP, RSRQ, RSSNR, RSSI) alongside GPS coordinates and cell identity fields in JSON format. Two separate measurement campaigns were conducted to capture spatial diversity.

Parameter	Zone 1	Zone 2	Unit
Raw rows	233,560	179,168	records
Registered records (filtered)	28,875	22,331	records
Unique towers (PCI)	11	15	—
Mean RSRP	-99.1	-92.2	dBm
Std RSRP	8.7	13.3	dB
RSRP Range	-128 to -67	-140 to -52	dBm
Mean UE-Tower Distance	156.3	99.3	m
Max Distance	632.0	490.2	m
Mean Path Loss	157.1	150.2	dB

Zone 2 shows a wider RSRP spread (σ = 13.3 dB vs 8.7 dB), more towers (15 vs 11), and shorter average distances, suggesting a denser urban environment with greater signal variability.

Zone 1 measurements are more homogeneous, likely corresponding to a more open or suburban area.

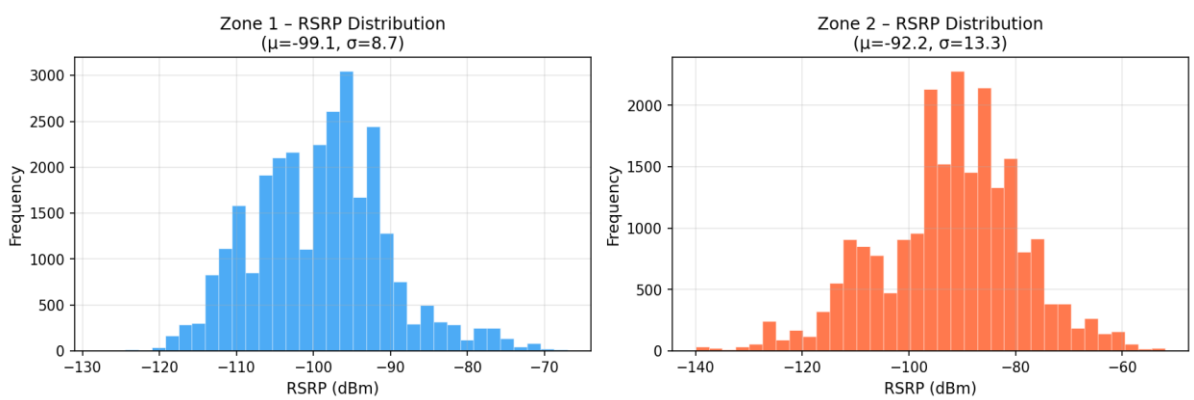

Figure 1 – RSRP distribution for Zone 1 (left) and Zone 2 (right) after filtering invalid readings.

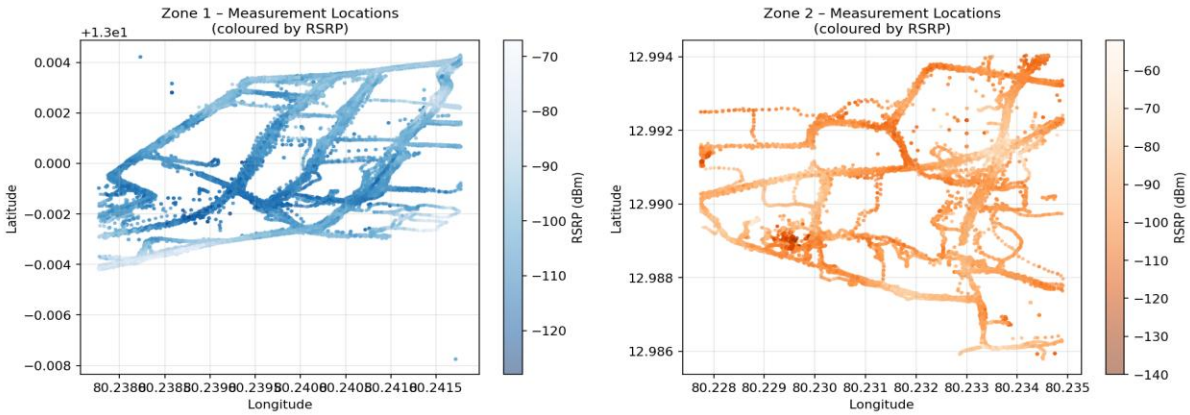

Figure 2 – Measurement GPS locations colour-coded by RSRP for Zone 1 (left) and Zone 2 (right).

3. Methodology

3.1 Data Preprocessing

- Only "registered" type records were retained (neighbour cells excluded).
- RSRP values were constrained to the valid range: −140 dBm to −44 dBm.
- Per-PCI mean GPS coordinates were used as tower locations.
- Haversine distance was computed from UE GPS to tower, and measurements within 20 m or beyond 3 km were excluded.

3.2 Feature Engineering

The following features were derived for model input:

- $\log_{10}$(distance) — linearises the power-law decay of signal strength.
- Δlat, Δlon — displacement vector from UE to tower (encodes direction).
- Angle (arctan2) — azimuth angle from tower to UE, capturing directional shadowing.
- K-Means cluster label (k=2) — captures dominant propagation environments.

3.3 Path Loss Computation

PL(d) = P_TX − RSRP where P_TX = 58 dBm

3.4 Models

Log-Distance (Baseline)

$PL(d) = PL_0 + 10 \cdot n \cdot \log_{10}(d)$ - fitted via ordinary least squares.

Random Forest & Gradient Boosting

Ensemble models trained directly on the 5 features to predict absolute path loss. Random Forest uses 200 trees with max depth 12; Gradient Boosting uses 300 estimators, learning rate 0.05, max depth 6.

Hybrid RF & Hybrid GB

The log-distance model first predicts PL. The residual (actual − log-distance prediction) is then modelled using an ensemble regressor. Final prediction = log-distance prediction + ML-predicted residual. This two-stage approach retains the physically-grounded trend while correcting for shadowing and local effects.

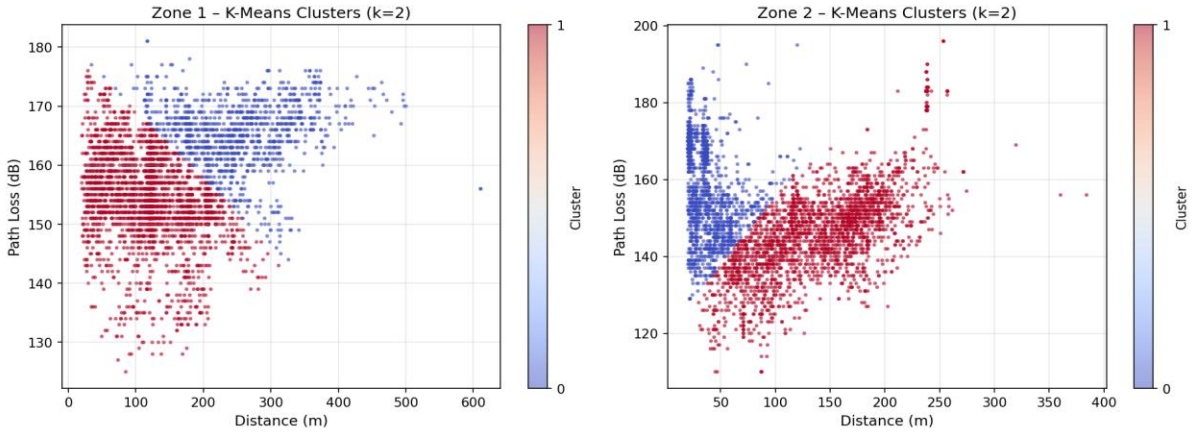

Figure 3 – K-Means (k=2) clustering of path loss vs distance. Cluster 0 (blue) and Cluster 1 (red) reveal two distinct propagation regimes.

4. Results & Analysis

4.1 Log-Distance Model Parameters

Parameter	Zone 1	Zone 2
PL_0 (dB)	Fitted via LSQ	Fitted via LSQ
Path Loss Exponent (n)	see Fig 4	see Fig 4
Baseline RMSE	8.51 dB	12.62 dB
Baseline R^2	0.040	0.057

The very low R^2 of the log-distance model (≈0.04–0.06) is expected in dense urban environments where shadowing variance dominates over the mean distance-dependent trend. The RMSE of 8.5–12.6 dB indicates this model alone is insufficient for practical network planning.

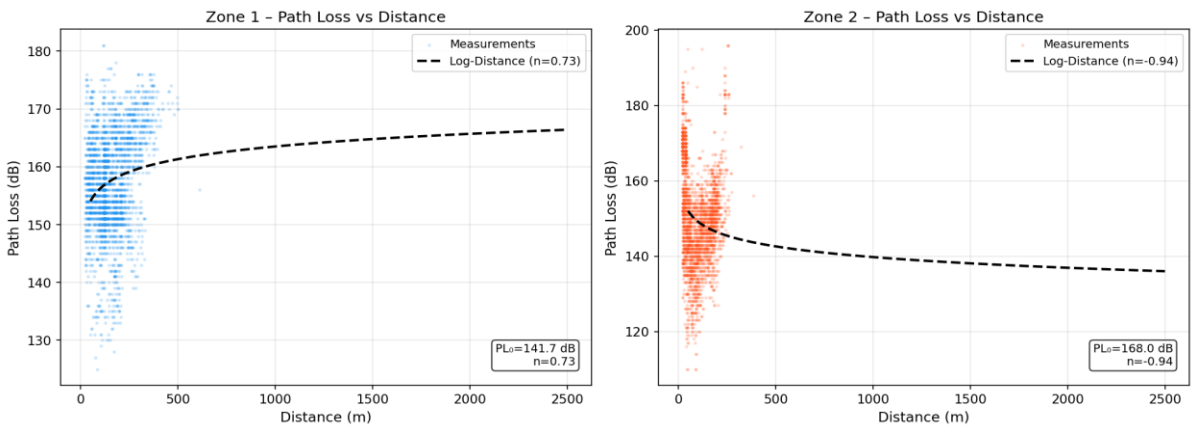

Figure 4 – Path loss scatter with fitted log-distance model curve. Zone 2 shows significantly higher spread.

4.2 Model Comparison — Zone 1

Model	RMSE (dB)	R^2	Rank
Log-Distance (Baseline)	8.51	0.0399	5
Random Forest	3.94	0.7943	2
Gradient Boosting	4.12	0.7751	4
Hybrid RF	**3.93**	**0.7957**	**1 ★**
Hybrid GB	4.14	0.7727	3

4.3 Model Comparison — Zone 2

Model	RMSE (dB)	R^2	Rank
Log-Distance (Baseline)	12.62	0.0570	5
Random Forest	**5.44**	**0.8249**	**1 ★**
Gradient Boosting	5.76	0.8032	3
Hybrid RF	5.54	0.8185	2
Hybrid GB	5.89	0.7948	4

ML models outperform the baseline by a 2.1×–2.3× reduction in RMSE. Zone 2's higher variability results in higher absolute RMSE across all models, but the ML models still achieve $R^2 > 0.79$ in both zones — a strong improvement over the log-distance baseline.

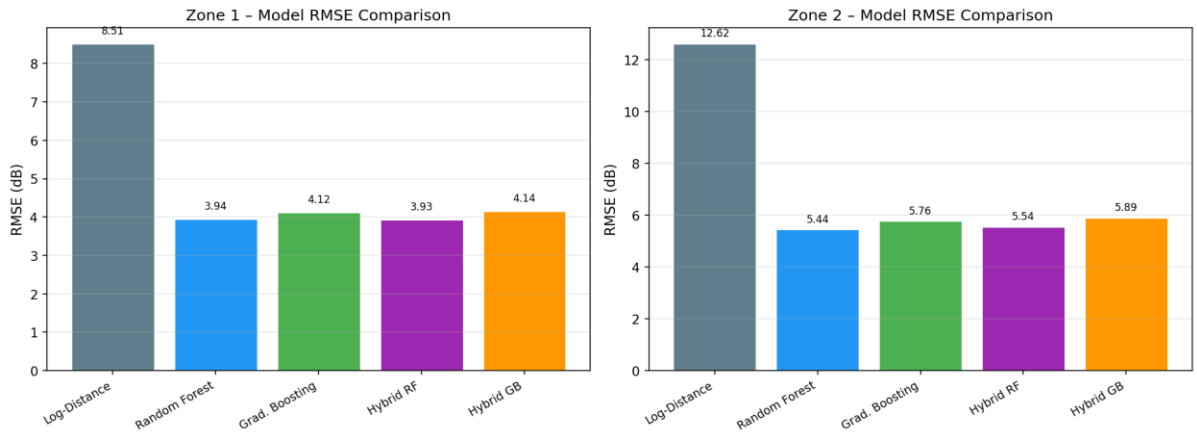

Figure 5 – RMSE comparison for all five models in Zone 1 (left) and Zone 2 (right).

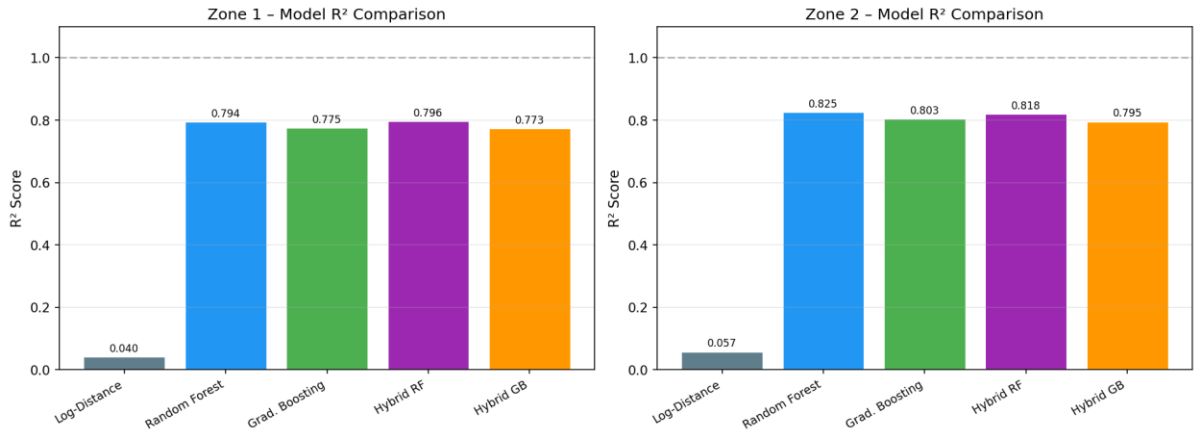

Figure 6 – R^2 score comparison for all five models.

4.4 Cross-Zone Comparison

The following chart directly compares model performance between zones, revealing how model accuracy scales with the complexity of the propagation environment.

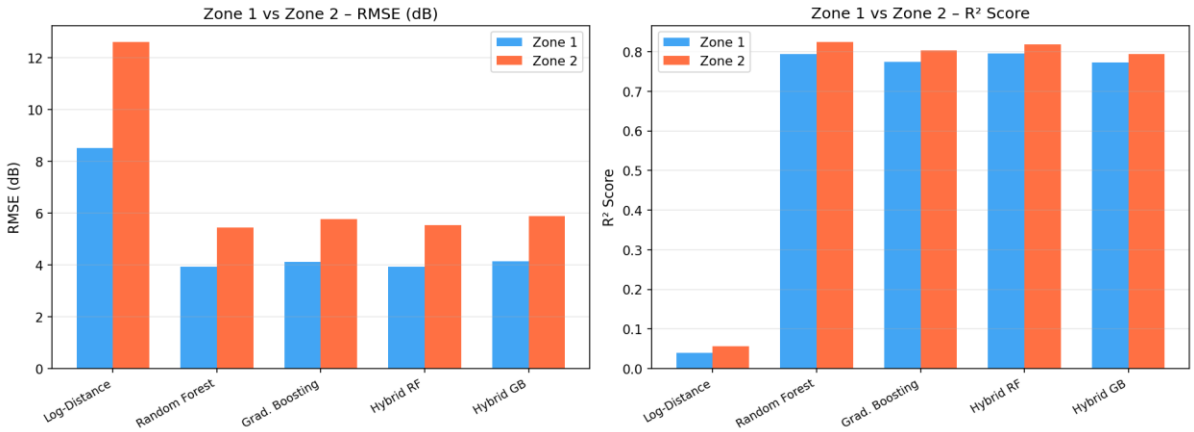

Figure 7 – Side-by-side RMSE and R² comparison across Zone 1 and Zone 2 for all models.

Key observations from the cross-zone analysis:

- Zone 1 consistently achieves lower RMSE (~1.5 dB lower) than Zone 2 for every model, reflecting its more homogeneous propagation environment.
- Zone 2 achieves slightly higher R² for ML models (0.82 vs 0.79), suggesting that ML features capture relative variance well despite higher absolute error.
- The Hybrid RF approach is the most consistent performer, ranking #1 in Zone 1 and #2 in Zone 2.

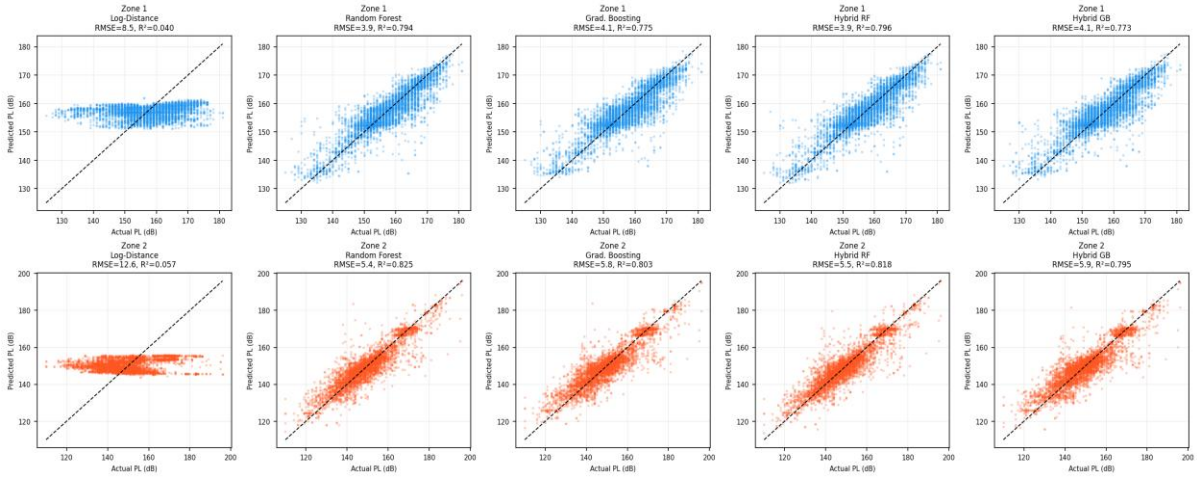

Figure 8 – Predicted vs Actual path loss scatter plots for all models across both zones. Ideal predictions lie on the 45° dashed line.

5. Conclusions

This study demonstrates that machine learning models significantly outperform classical empirical path loss models for cellular networks in urban environments. The key findings are:

- The Log-Distance model, while providing physical interpretability, achieves $R^2 \approx 0.04–0.06$ — insufficient for deployment-grade accuracy in complex urban terrain.
- Random Forest and Gradient Boosting regressors reduce RMSE by more than 50% compared to the baseline, achieving $R^2 > 0.79$ in both zones.
- The Hybrid Random Forest model (log-distance + ML residual) is the most consistent top performer, showing that combining physics-based priors with data-driven corrections yields the best results.
- Zone 2 has inherently higher path loss variability ($\sigma \approx 13.3$ dB) driven by its denser cell deployment and shorter measurement distances, leading to higher absolute RMSE across all models.
- Feature engineering (log-distance, angular features, K-Means cluster labels) provides the ML models with sufficient spatial context to model shadowing and environment-specific effects.

5.1 Future Work

- Extend the feature set to include building footprint data, land-use maps, and street-level morphology (3D GIS data).
- Replace K-Means cluster labels with semantically labelled environment types (dense urban, suburban, open area).
- Investigate deep learning approaches (LSTM for drive-test sequences, graph neural networks for cell topology).
- Evaluate XGBoost and LightGBM — expected to perform competitively with Gradient Boosting with faster training.
- Collect measurements across more zones and operators for cross-operator generalisation studies.

Code

Git Repo: https://github.com/cs25m525/WC_Project

JSON to CSV Conversion Code:

File name: jsonToCSV.py

```
import pandas as pd
import json
import csv
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

jsonFilePath = "C:/Dir/IITM-M.Tech/Wireless Communication/MiniProject/zone_cell_data/"
# jsonFileNameWithoutExtension = "zone1"
jsonFileNameWithoutExtension = "zone2"
jsonFileNameWithPath = jsonFilePath+jsonFileNameWithoutExtension+".json"

with open(jsonFileNameWithPath) as f:
    dataZone1 = json.load(f)

rows = []
# print(type(dataZone1))
# print(len(dataZone1))
# print(len(dataZone2))

#print(dataZone1[0]['GPS_data']['Latitude'])
#print(dataZone1[0]['registered']['Provider'])
# print(dataZone1[0]['neighbors'][0]['rsrp'])   #This is list of neighbors


# zoneOneData = pd.DataFrame(columns = ['Timestamp', 'Lat', 'Long', 'Type','Provider' ,'ci', 'pci', 'tac',
#                                        'earfcn', 'mcc', 'mnc', 'rsrp', 'rsrq', 'rssnr', 'rssi', 'level'])


zoneOneDataList = []   #Create empty List

for i in dataZone1:
    timestamp = i['Timestamp']
    lat = i["GPS_data"]["Latitude"]
    lon = i["GPS_data"]["Longitude"]
    registered = i['registered']
    #print(type(registered))
    localList = [timestamp, lat, lon, 'registered',registered.get('Provider'),
                                 registered.get('ci'),
                                 registered.get('pci'),
                                 registered.get('tac'),
                                 registered.get('earfcn'),
                                 registered.get('mcc'),
                                 registered.get('mnc'),
                                 registered.get('rsrp'),
```

```
                                registered.get('rsrq'),
                                registered.get('rssnr'),
                                registered.get('rssi'),
                                registered.get('level')]
    zoneOneDataList.append(localList)
    for j in i['neighbors']:
        localList = []
        localList = [timestamp, lat, lon, 'neighbour',j.get('Provider'),
                                j.get('ci'),
                                j.get('pci'),
                                j.get('tac'),
                                j.get('earfcn'),
                                j.get('mcc'),
                                j.get('mnc'),
                                j.get('rsrp'),
                                j.get('rsrq'),
                                j.get('rssnr'),
                                j.get('rssi'),
                                j.get('level')]
        zoneOneDataList.append(localList)


# Convert to DataFrame
zoneOneData = pd.DataFrame(zoneOneDataList, columns=['Timestamp', 'Lat', 'Long',
'Type','Provider','ci','pci','tac',
                                 'earfcn','mcc','mnc','rsrp','rsrq','rssnr','rssi','level'])

zoneOneData.head()
zoneOneData.to_csv(jsonFilePath+jsonFileNameWithoutExtension+".csv", index=False)
print("Zone Json Converted to CSV");
```

Path Loss Prediction Code

File Name: pathLossModel.py

```
##########################################################################################
#######################
# IMPORTS
##########################################################################################
#######################
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor

from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import mean_squared_error, r2_score

##########################################################################################
# LOAD DATA
# Data converted from json to csv using another python code.
#
##########################################################################################
print("Started...")

#Load data from csv file.. which was converted from json file(json to csv conversion was in a seperate
python code)
df = pd.read_csv('C:/Dir/IITM-M.Tech/Wireless Communication/MiniProject/zone_cell_data/zone2.csv')

#Dropping neighbour data as cell phone is not connected to it.
#It can create noise due to reflections
#Cell Phone only connected to the registered tower
#So to calculate path loss model we can are using registered type in our data
reg = df[df['Type'] == 'registered'].copy()
##########################################################################################
# ANALYZE DATA
##########################################################################################

#Checking spread of data using histogram
plt.figure()
plt.hist(reg['rsrp'], bins=30)
plt.xlabel("SNR (rsrp)")
plt.ylabel("Frequency")
plt.title("Histogram of SNR")
plt.savefig("C:/Dir/IITM-M.Tech/Wireless Communication/MiniProject/zone_cell_data/Histogram of
SNR.png")
plt.show()

#Removing rssnr as maximum values are 2147483647 which is int32 max value
#Removing rsrp also has some maximum values are 2147483647 which is int32 max value so dropping
it before building models
# reg = reg[reg['rsrp']  != 2147483647]
# reg = reg[reg['rssnr'] != 2147483647]

reg = reg[(reg['rsrp'] >= -140) & (reg['rsrp'] <= -44)].copy()

#Checking again spread of data using histogram
#Checking spread of data using histogram
plt.figure()
plt.hist(reg['rsrp'], bins=30)
plt.xlabel("SNR (rsrp)")
plt.ylabel("Frequency")
plt.title("Filtered SNR Histogram")
plt.savefig("C:/Dir/IITM-M.Tech/Wireless Communication/MiniProject/zone_cell_data/Filtered SNR
Histogram.png")
plt.show()
```

```
#Analyzing and Plotting lat Long data to check for any outliers
# print(reg['Lat'].describe())
# print(reg['Long'].describe())
plt.figure()
plt.scatter(reg['Long'], reg['Lat'])#, color='blue', marker='o')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.xlim(80.237700, 80.241800)
plt.ylim(12.992180, 13.004300)
plt.title("Location")
plt.grid(True)
plt.savefig("C:/Dir/IITM-M.Tech/Wireless Communication/MiniProject/zone_cell_data/Location.png")
plt.show()


##########################################################################################
# DISTANCE
##########################################################################################
def haversine(lat1, lon1, lat2, lon2):
    R = 6371000
    phi1, phi2 = np.radians(lat1), np.radians(lat2)
    a = (np.sin(np.radians(lat2 - lat1)/2)**2 +
        np.cos(phi1)*np.cos(phi2)*np.sin(np.radians(lon2 - lon1)/2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


# Assuming Base location from a reading which is near to the tower
bs_locs =
reg.groupby('pci')[['Lat','Long']].mean().rename(columns={'Lat':'bs_lat','Long':'bs_lon'}).reset_index()
reg = reg.merge(bs_locs, on='pci')

#Calulating distance of each row from the base location(lat, long)
reg['distance_m'] = haversine(reg['Lat'], reg['Long'], reg['bs_lat'], reg['bs_lon'])

#Avoided nearby signals
reg = reg[(reg['distance_m'] > 20) & (reg['distance_m'] < 3000)]

##########################################################################################
# PATH LOSS
##########################################################################################
#Assuming Power sent by Transmitter
P_TX = 58.0

#Calculating mathematical path loss
reg['path_loss_db'] = P_TX - reg['rsrp']

##########################################################################################
# FEATURES
```

```
##########################################################################################
#######################
# LOG-DISTANCE MODEL (mean path loss, no shadowing)
#   Mathematical Formulae PL(d) = PL0 + 10*n*log10(d)
#   Calculate change in lat and lon from base tower lat long
#   Calculate Angle to the tower using delta lat long
reg['log_distance'] = np.log10(reg['distance_m'])
reg['delta_lat'] = reg['Lat'] - reg['bs_lat']
reg['delta_lon'] = reg['Long'] - reg['bs_lon']
reg['angle'] = np.arctan2(reg['delta_lat'], reg['delta_lon'])

##########################################################################################
#######################
# CLUSTERING
##########################################################################################
#######################
#Applying k-means cluster algo
scaler = StandardScaler()
scaled = scaler.fit_transform(reg[['distance_m','path_loss_db']])
reg['cluster'] = KMeans(n_clusters=2, random_state=42).fit_predict(scaled)

##########################################################################################
#######################
# LOG DISTANCE
##########################################################################################
#######################
log_d = reg['log_distance'].values
pl = reg['path_loss_db'].values

X_fit = np.column_stack([np.ones_like(log_d), log_d])
pl0_fit, n10 = np.linalg.lstsq(X_fit, pl, rcond=None)[0]
n_fit = n10 / 10

##########################################################################################
#######################
# DATA SPLIT
##########################################################################################
#######################
# Features list, X,Y and Splitting dataset
features = ['log_distance','delta_lat','delta_lon','angle','cluster']
X = reg[features].values
y = reg['path_loss_db'].values

y_ld = pl0_fit + 10*n_fit*reg['log_distance']
residual = y - y_ld

X_train, X_test, y_train, y_test, res_train, res_test = train_test_split(
    X, y, residual, test_size=0.2, random_state=42
)

##########################################################################################
#######################
# MODELS
```

```
##########################################################################################
#######################

# Random Forest Algo.
rf = RandomForestRegressor(n_estimators=200, max_depth=12)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

# XGBoost Algo.
xgb = XGBRegressor(n_estimators=300, max_depth=6, learning_rate=0.05)
xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)

# Hybrid RF
rf_res = RandomForestRegressor(n_estimators=200, max_depth=12)
rf_res.fit(X_train, res_train)
y_pred_hybrid_rf = (pl0_fit + 10*n_fit*X_test[:,0]) + rf_res.predict(X_test)

# Hybrid XGB
xgb_res = XGBRegressor(n_estimators=300, max_depth=6, learning_rate=0.05)
xgb_res.fit(X_train, res_train)
y_pred_hybrid_xgb = (pl0_fit + 10*n_fit*X_test[:,0]) + xgb_res.predict(X_test)

# Log-distance
y_pred_ld = pl0_fit + 10*n_fit*X_test[:,0]

##########################################################################################
#######################
# COMPARISON METRICS
##########################################################################################
#######################
def evaluate(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true,y_pred)), r2_score(y_true,y_pred)

models = {
    "Log-Distance": y_pred_ld,
    "Random Forest": y_pred_rf,
    "XGBoost": y_pred_xgb,
    "Hybrid RF": y_pred_hybrid_rf,
    "Hybrid XGB": y_pred_hybrid_xgb
}

print("\nMODEL COMPARISON\n")
for name, pred in models.items():
    rmse, r2 = evaluate(y_test, pred)
    print(f"{name:<15} RMSE: {rmse:.2f}   R²: {r2:.4f}")

##########################################################################################
#######################
# GENERATE PLOTS
##########################################################################################
#######################
plt.figure(figsize=(10,6))
```

```
samp = reg.sample(min(5000,len(reg)))
plt.scatter(samp['distance_m'], samp['path_loss_db'], s=5, alpha=0.2)

d_curve = np.linspace(reg['distance_m'].min(), reg['distance_m'].max(), 500)
ld_curve = np.log10(d_curve)

plt.plot(d_curve, pl0_fit + 10*n_fit*ld_curve, label='Log-Distance')
plt.plot(d_curve, xgb.predict(np.column_stack([ld_curve, np.zeros((500,4))])), label='XGBoost')

plt.legend()
plt.xlabel("Distance")
plt.ylabel("Path Loss")
plt.grid()

plt.savefig("C:/Dir/IITM-M.Tech/Wireless
Communication/MiniProject/zone_cell_data/final_comparison_plot.png")
print("Plot saved.")
##################################################################################
#######################
```